CSS — Complete Study Notes

Beginner-friendly guide for first-year students | Concepts • Syntax • Examples • Best practices

Learning outcome: Learn to style HTML pages, arrange content with modern layouts, make pages responsive, add simple motion, and understand when CSS frameworks are useful.

1. Introduction to CSS

CSS stands for **Cascading Style Sheets**. It controls how HTML content looks: colors, fonts, spacing, borders, layout, and animations. HTML provides structure; CSS controls presentation.

Ways to add CSS

Method	How it works	When to use
Inline	CSS in an element's style attribute.	For a tiny one-off test; avoid for normal projects.
Internal	CSS inside a <style> element in the HTML head.	A single-page demonstration.
External	CSS in a separate .css file linked from HTML.	Best for most projects: reusable and maintainable.

External stylesheet:

```
<link rel="stylesheet" href="styles.css">
```

The browser reads HTML, loads the stylesheet, matches rules to elements, and displays the styled page.

2. Selectors, Properties, and Values

A CSS rule has a **selector** and a declaration block. Each declaration contains a property and a value.

```
selector {
  property: value;
}
```

Example:

```
p {
  color: navy;
  font-size: 18px;
}
```

• **Selector:** chooses which HTML element(s) to style — here, every paragraph.

• **Property:** what to change — for example, color or font-size.

• **Value:** the setting — for example, navy or 18px.

Common selectors

Selector	Targets	Example
Element	All elements of that type	p { color: black; }
Class (.name)	Elements with that class	.notice { padding: 10px; }
ID (#name)	The element with that ID	#header { background: gray; }
Universal (*)	Every element	* { box-sizing: border-box; }
Descendant	Matching element inside another	nav a { text-decoration: none; }
Group (comma)	Several selectors together	h1, h2 { color: navy; }

Class vs ID: classes are reusable and can be applied to many elements. An ID should be unique on a page. Prefer classes for reusable styling.

Cascade, specificity, and inheritance

• When multiple rules target an element, the cascade decides which declaration wins. Origin, importance, specificity, and source order matter.

• Specificity is selector strength: in general, an ID selector is stronger than a class selector, which is stronger than an element selector.

• Some properties, such as color and font-family, commonly inherit. Others, such as margin and border, normally do not.

• Avoid overusing !important; it makes styles harder to override.

3. Colors, Fonts, and Text Styling

Colors

CSS supports named colors, hexadecimal, RGB/RGBA, and HSL/HSLA values.

```
body { color: #222222; background-color: rgb(245, 247, 250); }
.card { border-color: hsl(210, 30%, 80%); }
```

• **color** sets text color; **background-color** sets the background.

• RGBA and HSLA include alpha transparency, for example rgba(0, 0, 0, 0.5).

Fonts and text

Property	Purpose	Typical values
font-family	Typeface	Arial, Georgia, sans-serif
font-size	Text size	16px, 1rem
font-weight	Thickness	400, 600, bold
font-style	Style	normal, italic
text-align	Alignment	left, center, right
text-decoration	Decoration	none, underline
line-height	Line spacing	1.5, 24px
letter-spacing	Space between letters	1px, 0.02em

Use readable contrast between text and background. A common starting point for body text is 16px with line-height around 1.4–1.6. Include fallback fonts.

4. CSS Box Model

Every element is treated as a rectangular box. The box model describes the space it occupies.

Box layers from inside to outside: Content → Padding → Border → Margin

• **Content:** text, image, or other material inside the element.

• **Padding:** space between content and border; it is inside the border.

• **Border:** the line surrounding padding and content.

• **Margin:** space outside the border, separating the element from neighbours.

Example:

```
.card {
  width: 300px;
  padding: 20px;
  border: 2px solid #333;
  margin: 16px;
}
```

With the default content-box model, total width is 300 + 40 + 4 = 344px, excluding margins. Use border-box to include padding and border within the declared width:

```
* { box-sizing: border-box; }
```

Vertical margins can collapse between some block elements; padding does not collapse.

5. Positioning and Display

Display

Value	Behaviour	Typical use
block	Starts on a new line and generally takes available width.	div, section, headings
inline	Flows within text; width/height do not work like block dimensions.	span, strong, links
inline-block	Flows inline but accepts width and height.	Badges or buttons
none	Removes the element from layout and accessibility tree in typical use.	Hide content completely
flex	Creates a one-dimensional flex container.	Navigation bars, aligned cards
grid	Creates a two-dimensional grid container.	Page sections, galleries

Position

• **static:** normal document flow; the default.

• **relative:** remains in flow but can be visually offset; can anchor absolutely positioned descendants.

• **absolute:** removed from normal flow and positioned relative to the nearest positioned ancestor, or the initial containing block.

• **fixed:** positioned relative to the viewport and usually stays in place while scrolling.

• **sticky:** behaves normally until a scroll threshold, then sticks within its scrolling container.

Offsets commonly use top, right, bottom, and left. z-index controls stacking order subject to stacking-context rules.

6. Flexbox and Grid Layouts

Flexbox — one-dimensional layout

Flexbox arranges items along one main axis: a row or a column. Use it for alignment, spacing, and flexible sizing.

```
.container {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
  flex-wrap: wrap;
}
```

• **flex-direction:** row or column sets the main axis.

• **justify-content:** distributes items along the main axis.

• **align-items:** aligns items along the cross axis.

• **gap:** adds space between items.

• **flex-wrap:** allows items to move onto another line.

Grid — two-dimensional layout

Grid controls rows and columns together. It is useful for page layouts, dashboards, and galleries.

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

fr is a fractional unit that distributes available space. **repeat()** repeats a track definition.

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

Flexbox is a good fit when…	Grid is a good fit when…
Items form a row or column and alignment is the main concern.	You need control over rows and columns at the same time.

Flexbox is a good fit when…	Grid is a good fit when…
You are arranging a navigation bar or button group.	You are arranging a page layout, dashboard, or card gallery.

7. Responsive Design and Media Queries

Responsive design makes a website work across different screen sizes. Build flexible layouts rather than assuming every screen has the same width.

• Use the viewport meta tag so mobile browsers size the page correctly.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

• Use flexible widths such as %, max-width, fr, and minmax() where suitable.

• Use responsive images: max-width: 100%; height: auto.

• Choose breakpoints when the layout needs to change, not just for device names.

Media query example

```
.content { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }

@media (max-width: 700px) {
  .content { grid-template-columns: 1fr; }
}
```

The media query applies when the viewport is 700px wide or narrower. This changes a two-column layout into one column.

Mobile-first approach: write base styles for small screens, then add min-width media queries for larger screens.

8. CSS Animations and Transitions

Transitions

A transition smoothly changes a property from one value to another when a state changes, often on hover or focus.

```
.button {
  background-color: #245b78;
  transition: background-color 0.25s ease, transform 0.25s ease;
}
.button:hover {
  background-color: #173d5b;
  transform: translateY(-2px);
}
```

A transition defines the property, duration, timing function, and optional delay. Keep effects subtle and provide visible keyboard focus styles.

Animations with @keyframes

Animations can run automatically and define multiple stages using keyframes.

```
@keyframes fade-in {
  from { opacity: 0; transform: translateY(8px); }
  to   { opacity: 1; transform: translateY(0); }
}
.card { animation: fade-in 0.5s ease-out both; }
```

• animation-name selects the keyframes; animation-duration sets one cycle's length.

• animation-timing-function controls the speed curve.

• animation-iteration-count sets the number of cycles, for example 1 or infinite.

Respect reduced-motion preferences:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

9. CSS Frameworks: Bootstrap and Tailwind

A CSS framework provides ready-made styles, layout tools, or utility classes. Frameworks can speed up development, but they do not replace CSS fundamentals.

Bootstrap

Bootstrap is a component-oriented front-end framework. It provides a responsive grid, predefined components, and utility classes.

```
<div class="container">
  <div class="row">
    <div class="col-md-6">First column</div>
    <div class="col-md-6">Second column</div>
  </div>
</div>
```

• Useful for consistent, ready-to-use components such as buttons, navigation bars, and cards.

• Its default visual style may need customisation for a unique design.

Tailwind CSS

Tailwind CSS is a utility-first framework. You compose designs by combining small classes directly in markup.

```
<div class="flex items-center gap-4 p-4 rounded-lg shadow">
  <h2 class="text-xl font-bold">Welcome</h2>
</div>
```

• Useful for building custom designs quickly without writing a separate CSS class for every component.

• Markup can become crowded if class combinations are not organised.

Feature	Bootstrap	Tailwind CSS
Approach	Components and utility classes	Utility-first classes
Starting point	Pre-designed components and grid	Low-level styling utilities
Custom look	Customise component styles	Compose utilities to create the design
Good fit	Fast standard layouts and familiar components	Highly customised interfaces

10. Best Practices

• Keep CSS in external stylesheets for most projects and organise rules by component or page section.

• Use meaningful, reusable class names such as .product-card.

• Prefer classes for styling; avoid unnecessary ID selectors and overly specific selector chains.

• Use consistent spacing, typography, and a small set of design colours.

• Use box-sizing: border-box and test layouts at narrow and wide viewport sizes.

• Choose Flexbox or Grid based on the layout problem; avoid excessive positioning.

• Maintain readable colour contrast, visible keyboard focus, and semantic HTML.

• Avoid unnecessary !important declarations and duplicated rules.

• Use comments for non-obvious decisions and remove unused CSS when practical.

• Test in more than one browser and check for horizontal scrolling on mobile.

11. Mini Practice: Style a Simple Card

HTML:

```
<div class="card">
  <h2>Learn CSS</h2>
  <p>Make web pages clear and attractive.</p>
  <a href="#">Read more</a>
</div>
```

CSS:

```
* { box-sizing: border-box; }
body { font-family: Arial, sans-serif; background: #f4f6f8; }
.card {
  max-width: 320px; margin: 40px auto; padding: 20px;
  background: white; border: 1px solid #d8e0e7;
  border-radius: 12px; box-shadow: 0 4px 14px rgba(0,0,0,.08);
}
.card h2 { color: #173d5b; }
.card a { color: #245b78; text-decoration: none; }
.card a:hover { text-decoration: underline; }
```

Experiment by changing the background, padding, border radius, font size, and shadow. Then make the card responsive and add a transition.

12. Quick Revision and Practice Questions

Question	Key point
What does CSS stand for?	Cascading Style Sheets; it controls HTML presentation.
What are the parts of a CSS declaration?	Property and value, separated by a colon.
What is the box model order?	Content, padding, border, margin.
Flexbox vs Grid?	Flexbox is mainly one-dimensional; Grid is two-dimensional.
What does a media query do?	Applies CSS conditionally, for example at a viewport width.
Transition vs animation?	Transitions respond to state changes; animations use keyframes.
Bootstrap vs Tailwind?	Bootstrap offers ready-made components; Tailwind focuses on utility classes.

Practice tasks

- Create a page with a heading, paragraph, image, and button. Style them using an external CSS file.
- Make three cards align in a row using Flexbox, then recreate the layout using Grid.
- Use a media query to stack the cards on a narrow screen.
- Add a hover transition to the button and a keyframe animation to the heading.
- Rebuild the card using Bootstrap or Tailwind utility classes.

Suggested learning order: HTML basics → CSS syntax and selectors → colors and typography → box model → display and positioning → Flexbox → Grid → responsive design → transitions and animations → frameworks.

End of notes • Practise by changing values in your own stylesheet.